Middleware Architecture (IS3108/SCS3203)

Scenario-Based Class Assessment - 2025

Duration - 2 Hours (Answer all 4 Questions)

Assignment Scenario - SriBank PLC

SriBank PLC is (SriBank) a Sri Lankan-based commercial bank that is into both corporate and retail banking products and services. The bank has operated for over 20 years with more than 100 branches across the island. SriBank is considered one of the most technologically superior customer-centric organisations in the country.

SriBank implemented the core banking solution during the initial years and is currently, connected to all its branches in an online manner. The bank had also implemented a 500 ATM (Automated Teller Machines) network to extend its reach to its retail customers. The customers are also given the facility to use credit and debit cards certified by MASTER and VISA international payment networks.

The management of SriBank has strategised to implement an Internet and Mobile Banking solution with the objective of enhancing customer access to the bank's services through digital channels. The key functionalities expected out of the proposed Internet and Mobile Banking Platform are as follows.

1. The types of primary transactions that should be accommodated by this system are balance inquiry, account summary, funds transfer, utility bill payments, online account creation/sign-up, etc.

2. The solution should accommodate the needs of only retail banking customers (Corporate Banking should be considered out of scope). Retail customers should be properly authenticated (two-factor) for access to the platform.

3. The special service requests of the new Internet and Mobile Banking Platform are the ability to create new accounts, request loans and request special service approvals like credit

limit increments, etc. However, these types of workflow-based transactions may not be online and real-time when compared to other types such as balance inquiry, funds transfer, etc. These special types of transactions may need workflow approvals from several departments/personnel that interact through different systems such as credit management, core banking, etc.

4. The Internet and Mobile Banking Platform should integrate with the following systems to provide overall functionality.

    a. Core Banking System - maintains the customer profiles (KYC) and account information (balance, transaction history, interest, etc). The integration is based on RESTful services.

    b. Inter-bank Payment Gateway (e.g. CEFT, JustPay) - Provided by the Central Bank of Sri Lanka for routing payment requests across participating banks in Sri Lanka. The integration is based on ISO 8583 specifications.

    c. Credit Card System - The credit card information of customers such as credit card limits, balance, active status, billing cycle, minimum payment, etc, are maintained. The integration is based on SOAP standards.

    d. Internal Payment Switch - The payment switch links external international payment processing providers such as MasterCard and VISA. The integration is based on TCP/IP-based POX (Plain Old XML) interface.

The new platform for providing Internet and Mobile Banking should integrate with the above-mentioned systems. Mobile Banking should enable the use of iOS and Android platforms and Internet Banking should work on standard Internet browsers and the interfaces should be mobile responsive. The business logic for both mobile and internet banking should be common and amalgamated. The customer and other information should be persisted in a relational database. The mobile banking solution should provide notifications to the user's mobile application as push notifications as well as in the mode of SMS.

Question 1 :

Based on the SriBank PLC scenario, **design and propose a comprehensive software architecture** for the new Internet and Mobile Banking Platform.

Your proposal should:

a. Clearly identify and describe the logical **layers** and **tiers** of your proposed architecture. Justify your choice of architectural tiering.

b. For each layer and tier, list and briefly describe the key **components** you would envision.

c. **Discuss and justify** the application of **at least three (3) distinct architectural patterns** within your proposed overall solution. Explain how each pattern contributes to meeting the system's requirements (e.g., scalability, maintainability, security, integration).

d. Provide a clear and well-labeled **architectural diagram** illustrating the tiers, layers, components, and primary communication flows.

[Marks 25]

Note: Any assumptions taken should be clearly mentioned.

Answer 1 :

Based on the SriBank PLC scenario, a robust and scalable **Multi-Tiered Architecture** integrated with **Service-Oriented Architecture (SOA)** principles is the most suitable approach for the new Internet and Mobile Banking Platform. This combination provides clear separation of concerns, flexibility for diverse clients, efficient integration with heterogeneous backend systems, and supports the critical non-functional requirements of a banking application like security, scalability, and maintainability.

a. Logical Layers and Physical Tiers:

It's crucial to distinguish between logical layers and physical tiers:

- **Tiers:** Refer to the logical organisation of code, separating functionalities into distinct groups based on responsibility (e.g., Presentation, Business Logic, Data Access). These are software constructs.
- **Layers:** Each tier to be segmented into logical layers to separate the functionality of a tier.

For SriBank PLC, we will propose a **4-Tier Architecture** to accommodate the distinct needs of client applications, core business logic, complex integrations, and data persistence.

1. **Client/Presentation Tier:**
    - **Logical Tier:** Presentation Layer.
    - **Purpose:** Responsible for user interaction, displaying information, capturing user input, and performing client-side validations. It provides the user interface for the Internet and Mobile Banking services.
    - **Justification:** Separating this tier allows for different client technologies (web browsers, iOS apps, Android apps) to be developed independently, optimised for their respective platforms, and scaled based on user access patterns, without affecting the core business logic. This directly addresses the requirement for both Internet and Mobile Banking interfaces.
2. **Application/Business Logic Tier:**
    - **Logical Tier:** Business Logic Layer, Service Layer.
    - **Purpose:** Houses the core business rules and functionalities (e.g., funds transfer logic, account summary processing, special service request initiation). It acts as a facade, exposing a unified API to the Presentation Tier, orchestrating calls to internal and external services, and handling application-level security (authenti-

cation, authorisation). It ensures that the business logic is common and amalgamated for both web and mobile clients.

- **Justification:** Centralising business logic here ensures consistency across all digital channels, promotes reusability, simplifies maintenance, and provides a crucial security boundary between the public-facing clients and sensitive backend systems. It allows independent scaling of business logic without impacting the data or integration systems.

3. **Integration Tier:**

- **Logical Layer:** Integration Layer, Data Access Layer (for external systems).
- **Purpose:** Acts as a mediation and translation layer between the Business Logic Tier and the diverse existing backend systems. It handles protocol conversions, data transformations, message routing, and ensures secure, standardised communication with heterogeneous interfaces (REST, ISO 8583, SOAP, POX). It also facilitates asynchronous communication for workflows and notifications.
- **Justification:** This tier is critical due to SriBank's existing heterogeneous systems. It decouples the core business logic from the complexities of legacy system interfaces, making the overall architecture more resilient to changes in backend systems and easier to manage. It centralises integration concerns, improving maintainability and reducing direct dependencies.

4. **Data Tier:**

- **Logical Layer:** Data Storage Layer.
- **Purpose:** Manages the persistence of data specific to the new Internet and Mobile Banking platform (e.g., user preferences, session data, audit logs, workflow states). It represents the physical location of both the new platform's database and the external databases/systems (Core Banking, Credit Card, Payment Gateways) that hold core banking data.
- **Justification:** Separating the Data Tier provides specialised management of data, allowing for optimised database performance, robust transaction management (ACID properties), data security, and independent scaling of storage resources. It ensures that data remains consistent and available.

b. Key Components of Each Layer and Tier:

1. **Client/Presentation Tier Components:**
   - **Internet Banking Web Application:**
     - **UI Framework:** Modern JavaScript framework (e.g., React, Angular, Vue.js) for building a responsive Single Page Application (SPA).
     - **HTML/CSS/JavaScript Assets:** Files served to the browser.
     - **Client-Side Validation Logic:** JavaScript code for immediate user input validation.
     - **API Client Library:** Code to interact with the Application/Business Logic Tier's RESTful APIs.
   - **Mobile Banking App:**
     - **Native iOS UI Components:** Developed using Kotlin/Java,Swift/Objective-C, Flutter.
     - **API Client Library:** To consume RESTful APIs from the Application/Business Logic Tier.
     - **Push Notification Handler:** Logic to receive and display push notifications (APNS).
     - **Local Data Storage:** For caching or offline capabilities where appropriate.
2. **Application/Business Logic Tier Components:**
   - **API Gateway:**
     - **Request Router:** Directs incoming requests to appropriate internal services.
     - **Authentication & Authorisation Module:** Validates user credentials (including 2FA) and access tokens, enforces access control policies.
     - **Rate Limiter/Throttler:** Protects backend services from overload.
     - **Logging & Monitoring Agent:** Collects API usage metrics and logs.
   - **Backend for Frontend (BFF) / Business Logic Microservices (or modular services):**

        - **Retail Banking Service:** Implements core functions like Balance Inquiry, Account Summary, Funds Transfer (orchestrates calls to Integration Tier), Utility Bill Payments.
        - **Account Management Service:** Handles online self-registration for *platform access*, customer profile management.
        - **Special Service Request Service:** Initiates workflow-based requests (e.g., Loan Requests, Credit Limit Increments, new full account creation). Publishes events to the message broker.
        - **Security & Identity Service:** Manages user registration, password policies, 2FA enrolment/verification, session management, and JWT token issuance.
        - **Notification Orchestration Service:** Exposes an API for other internal services to trigger notifications.
        - **Common Utility Components:** (e.g., centralised logging, configuration management, error handling).

3. **Integration Tier Components:**
    - **Message Broker (e.g., Kafka, RabbitMQ):**
        - **Topics/Queues:** For asynchronous communication (e.g., notification_events, workflow_request_events, workflow_status_updates).
    - **Integration Adapters/Proxies:**
        - **Core Banking System Adapter:** Translates internal requests into RESTful calls for the Core Banking System and responses back to the internal format.
        - **Inter-bank Payment Gateway Adapter:** Formats internal payment requests into ISO 8583 messages for the Central Bank's gateway, handles parsing responses.
        - **Credit Card System Adapter:** Handles SOAP message construction and parsing for communication with the Credit Card System.
        - **Internal Payment Switch Adapter:** Manages TCP/IP-based POX communication and data mapping for MasterCard/VISA processing.
    - **Workflow Engine:**

        - **Process Orchestrator:** Manages the state and flow of complex, multi-step special service requests (loan approvals, new account creation workflows), interacting with relevant internal and external systems via adapters.
        - **Task Management:** Assigns tasks to relevant departments/personnel.
    - **Notification Delivery Service:**
        - **Push Notification Gateway Integrator:** Interfaces with APNS (Apple) and FCM (Google) for mobile push notifications.
        - **SMS Gateway Integrator:** Interfaces with an SMS service provider for text messages.
        - **Notification Templating Engine:** Formats notification content.

4. **Data Tier Components:**
    - **Relational Database Management System (RDBMS):**
        - **Database Server:** Hosts the database instance.
        - **Application Database:** Stores data specific to the new platform (e.g., user profiles for digital access, notification preferences, audit trails for digital transactions, workflow request states, user sessions).
    - **Existing External Systems (Accessed via Integration Tier):**
        - **Core Banking System Database:** (Holds customer KYC, account balances, transaction history, interest info).
        - **Inter-bank Payment Gateway:** (Routes payment requests).
        - **Credit Card System Database:** (Holds credit card limits, balances, status).
        - **Internal Payment Switch:** (Links to international payment processors).

c. Discussion and Justification of Architectural Patterns in SriBank PLC's Solution

The proposed overall architecture for SriBank PLC's Internet and Mobile Banking Platform leverages several distinct architectural patterns to address its diverse requirements, including scalability, maintainability, security, and integration with existing heterogeneous systems. We

will discuss and justify the application of **3-Tier Architecture**, **Service-Oriented Architecture (SOA)**, **Publish-Subscribe**, and **Model-View-Controller (MVC)**.

1. 3-Tier (or N-Tier) Architecture

- **Description:** The 3-Tier (or more generally N-Tier) architectural pattern logically and often physically separates an application into distinct layers, each with a specific responsibility. The classic three tiers are:

    1. **Presentation Tier:** Handles the user interface and user interaction.

    2. **Application/Business Logic Tier:** Contains the core business rules and processes.

    3. **Data Tier:** Manages data storage and retrieval. This separation promotes modularity and allows for independent development, deployment, and scaling of each tier.

- **Justification & Contribution to Requirements:**

    1. **Scalability:** Each tier can be scaled independently based on its specific load. For SriBank, if mobile app usage surges, the **Presentation Tier** (web servers, mobile app backend for frontend) can be scaled horizontally without necessarily needing to scale the entire **Data Tier**. This optimises resource utilisation and cost.

    2. **Maintainability:** Changes or updates within one tier typically have minimal impact on others. For example, enhancing the user interface in the **Presentation Tier** does not require changes to the core business logic in the **Application Tier**. This significantly simplifies development, testing, and deployment cycles, which is crucial for a continuously evolving banking product.

    3. **Security:** This layered approach provides inherent security benefits by creating clear demarcation points and enforcing strict communication rules between tiers. The **Application/Business Logic Tier** can act as a crucial security boundary (a DMZ), preventing direct client access to the sensitive **Data Tier** and legacy systems in the **Integration Tier**. This multi-layered defence makes the system more robust against attacks.

    4. **Flexibility:** It allows for the use of different technologies optimised for each tier. For instance, native mobile development for apps, a robust Java/Spring Boot backend for business logic, and a high-performance relational database for data persistence. This flexibility supports SriBank's need for both mobile and internet banking on diverse platforms.

2. Service-Oriented Architecture (SOA)

- **Description:** SOA is an architectural paradigm that structures an application as a collection of loosely coupled, interoperable, and reusable services. Each service encapsulates a specific business capability, exposes a well-defined interface, and can be discovered and invoked by other services or applications. Communication often occurs via standard protocols like REST or SOAP.

- **Justification & Contribution to Requirements:**

    - **Integration:** This is arguably SOA's most critical contribution to the SriBank scenario. The platform needs to integrate with existing heterogeneous systems (Core Banking via REST, Inter-bank Payment Gateway via ISO 8583, Credit Card System via SOAP, Internal Payment Switch via TCP/IP POX). SOA enables this by creating an **Integration Tier** composed of dedicated service adapters that abstract away the complexities of these diverse protocols. The Retail Banking Service (in the Application Tier) can simply call a generic "funds transfer service" without needing to know if it's hitting a REST, SOAP, or ISO 8583 backend.

    - **Reusability:** Core banking functionalities (e.g., FundsTransfer, AccountSummary, Authentication) are exposed as distinct services. These services are then reusable across both the Internet Banking and Mobile Banking applications, ensuring consistent business logic and reducing redundant development. Future applications within SriBank could also consume these same services.

    - **Maintainability & Flexibility:** Services can be developed, deployed, and updated independently. If the logic for "Utility Bill Payments" changes, only that specific service needs modification and redeployment, minimising disruption to other parts of the system. This modularity allows for quicker adaptation to evolving business rules and easier bug fixes.

    - **Scalability:** Individual services can be scaled independently based on their specific demand. A highly utilised "Balance Inquiry" service can be scaled more aggressively than a less frequent "Loan Request" service, optimising resource allocation.

3. Publish-Subscribe (Pub/Sub) Architecture

- **Description:** Pub/Sub is a messaging pattern where "publishers" send messages to a "message broker" without knowing who the recipients ("subscribers") are. Subscribers express interest in specific message types (topics) and receive messages published to

those topics. This pattern achieves strong decoupling between message producers and consumers.

- **Justification & Contribution to Requirements:**
    - **Reliable Notifications:** For crucial notifications (push, SMS) like transaction confirmations, Pub/Sub is ideal. When a transaction is completed, the core banking service (publisher) publishes a TransactionCompletedEvent to the **Message Broker**. A dedicated Notification Delivery Service (subscriber) asynchronously picks up this event and handles the actual sending. If the notification service is temporarily down or mobile network conditions are volatile, the message remains in the broker's queue, ensuring "at-least-once" delivery and eventual reliability without blocking the core banking transaction.
    - **Asynchronous Workflow for Special Requests:** The scenario states "special service requests" (loans, credit limit increments) are not real-time and require workflow approvals. A Workflow Engine (subscriber) picks this up to begin the multi-departmental approval process. Status updates from the workflow can also be published, decoupling the request initiation from the long-running approval process.
    - **Scalability:** The Message Broker acts as a buffer, smoothing out peaks in message traffic. The Notification Delivery Service (and other subscribers) can be scaled horizontally by adding more instances to consume messages in parallel, increasing throughput as notification volume grows. This prevents notification processing from becoming a bottleneck for the core banking system.
    - **Decoupling:** Publishers don't need to know the specifics of how notifications are delivered (APNS, FCM, SMS). They simply publish an event. This makes the system more resilient to changes in delivery mechanisms or the addition of new notification channels, enhancing maintainability.

4. Model-View-Controller (MVC) Architecture (Client-Side Focus)

- **Description:** MVC is an architectural pattern that separates an application into three main components:
    - **Model:** Manages the application's data, business rules, and state.
    - **View:** Responsible for displaying the user interface (UI) based on the Model's data.

- **Controller:** Handles user input, processes it, updates the Model, and orchestrates View updates. While originally a server-side pattern, it's widely applied client-side (often as part of a Single Page Application framework).

- **Justification & Contribution to Requirements:**
    - **Exceptional User Responsiveness & Experience (Client-Side):** This is MVC's primary contribution to the Internet Banking front-end. By separating concerns, MVC allows for rich client-side interactions. The Controller manages user input locally, updating the View dynamically without full page reloads, making the application feel fast and fluid like a desktop app.
    - **Client-Tier Responsibility for Validations:** The **Model** can encapsulate client-side validation rules. When the **Controller** receives user input from the **View**, it can perform instant validation against the Model's rules ***before*** sending data to the server. For example, checking email format or minimum password length immediately. This provides instant feedback to the user, reducing errors and improving the overall user journey significantly.
    - **Maintainability of UI Logic:** The clear separation of concerns makes the front-end code easier to understand, develop, and maintain. Developers can work on UI rendering (View) independently from data handling (Model) or user interaction logic (Controller). This modularity is essential for complex banking UIs.
    - **Testability:** Each component of MVC can be tested in isolation. For example, the Model's validation logic can be unit-tested without needing a UI, which improves the quality and reliability of the client-side application.

By strategically combining these architectural patterns, the SriBank PLC Internet and Mobile Banking Platform can effectively address its diverse functional and non-functional requirements, ensuring a scalable, maintainable, secure, and user-friendly solution.

d. Architecture diagram

Give marks to the architecture diagram based on the components in section (b) and architectural patterns in section (c).

Question 2:

The modern browser-based Internet Banking application requirements may emphasise client-tier extended responsibilities such as input validation, reactive user experience/responsiveness, rather than focusing on traditional presentation tier functionality.

a. Identify and describe the most suitable **client-side architectural pattern** for this scope.
b. Explain, with suitable examples, how this chosen pattern enables **client-side validation** and contributes to **better user responsiveness and experience** as stipulated in the scenario.
c. Briefly discuss any significant **architectural considerations or trade-offs** associated with implementing this pattern in a banking context (e.g., security implications, initial load times).

[Marks 25]

Answer 2:

a. Description of the client-side architectural pattern:

For the browser-based Internet Banking applications, where client-tier responsibility for input validation and an exceptional user experience is paramount, the **Client-Side Model-View-Controller (MVC)** architectural pattern is highly suitable. This pattern, or its modern variations like Model-View-ViewModel (MVVM) or Model-View-Presenter (MVP), provides a structured way to build interactive Single Page Applications (SPAs) that deliver on these requirements.

b. How Client-Side MVC Enables Client-Side Validation and Enhanced User Experience:

Client-Side MVC, when applied to a browser-based banking platform, significantly enhances both validation and user experience:

- **Enabling Client-Side Validation:**
    - **Model-Driven Validation:** The **Model** can contain validation rules directly related to the data it manages. For instance, an Account model might have rules for valid account numbers, or a Transaction model might enforce rules for positive amounts and valid currency codes. When user input is received by the Controller and attempts to update the Model, these validation rules are instantly checked within the client's browser.
    - **Controller as Orchestrator:** The **Controller** receives user input from the **View** (e.g., a user types into an input field). Before attempting to update the Model or send data to the server, the Controller can trigger validation checks against the Model's rules or its dedicated validation logic.
    - **View for Immediate Feedback:** If validation fails, the Controller instructs the **View** to display immediate, contextual error messages next to the offending input fields. This provides instant feedback without any network delay, allowing the user to correct mistakes quickly. For example, if a user tries to transfer a negative amount, the Controller catches it, prevents Model update, and the View immediately shows "Amount must be positive."
    - **Example (Funds Transfer):**
        - **View:** Displays the input fields for 'beneficiary account number' and 'amount'.
        - **Controller:** On user input, the Controller retrieves the value. It then asks the **Model** to validate the format of the account number. It also checks if the amount entered is a valid number and within a pre-defined client-side range (e.g., not zero).

        - **Model:** Contains regex patterns for account numbers and constraints for amounts.
        - **View (Update):** If validation fails, the Controller signals the View to display a red border and a message like "Invalid Account Number Format" or "Amount cannot be zero." This happens synchronously within the browser, providing a highly responsive experience.

- **Contributing to Better User Responsiveness and Experience:**
    - **Separation of Concerns:** MVC's clear separation ensures that the UI (View) can be updated independently of the data and business logic (Model). This allows for efficient partial updates of the page.
    - **Reduced Server Roundtrips:** After the initial load of the SPA (which is typically built using MVC principles), subsequent user interactions often only require fetching or sending small data payloads (JSON) to the backend API, orchestrated by the Controller. The View then re-renders only the necessary parts of the DOM. This eliminates full page reloads, making navigation and interactions feel instantaneous.
    - **Responsive UI:** Since much of the application logic and rendering happens client-side, the UI feels snappier and more interactive. Users don't wait for server-side page generation for every click. This translates to a fluid experience crucial for banking, where users expect quick responses for balance checks, transaction history, and form submissions.
    - **State Management:** The Model centralises the application's state, making it easier to manage and keep the UI consistent. If a user's balance changes (after a funds transfer), the Model updates, notifying the relevant Views (e.g., Account Summary, Dashboard), which then automatically re-render, ensuring the user always sees up-to-date information without manual refreshes.
    - **Example (Account Summary):**
        - **Controller:** Fetches account data from the backend API.
        - **Model:** Stores the retrieved account data.
        - **View:** Renders the account summary table.
        - **User Action:** User clicks on "View Transactions" for a specific account.
        - **Controller:** Fetches transaction data for that account.
        - **Model:** Updates with transaction data.

- **View (Update):** Only the transaction list area of the page is updated dynamically, potentially with a smooth animation, without reloading the entire page. This provides a seamless transition.

c. Architectural Considerations and Trade-offs in a Banking Context:

Implementing Client-Side MVC for a banking platform, while beneficial, presents specific trade-offs:

1. **Security Implications:**
    - **Client-Side Vulnerabilities:** Any code executed in the browser is susceptible to inspection and manipulation. While client-side MVC enhances UX, it **does not replace server-side security**. All validation performed on the client must be rigorously repeated on the server before any data is processed or persisted. This means maintaining two sets of validation rules (one for UX, one for security/integrity), which can add overhead.
    - **API Security:** The client-side application relies entirely on consuming APIs from the Application/Business Logic Tier. Robust API security (authentication, authorisation, input sanitisation, rate limiting) is paramount. Any sensitive data should *never* be stored unencrypted in browser storage (Local Storage, Session Storage), as these are vulnerable to XSS attacks. Strategies like short-lived access tokens and HTTP-only cookies for refresh tokens are crucial for token management.
    - **Exposure of Business Logic (Partial):** While core business logic resides server-side, aspects of presentation logic and validation rules embedded in the client-side Model or Controller are viewable by anyone inspecting the browser's source code. No truly sensitive logic should *ever* reside exclusively on the client.
2. **Initial Load Times:**
    - **Larger Initial Bundle:** The entire MVC structure (JavaScript for Model, View, Controller, framework code, dependencies) for the SPA needs to be downloaded on the first visit. This can result in a larger initial payload and slower "time to interactive" compared to traditional server-rendered applications, especially for users with slower connections or older devices.
    - **Mitigation:** Techniques like **code splitting**, **lazy loading of modules**, **tree shaking**, and **aggressive caching** of static assets are essential to reduce the perceived load time and optimise delivery of the SPA's JavaScript bundle.
3. **Increased Client-Side Complexity:**

- Building a well-structured Client-Side MVC application requires more sophisticated JavaScript development skills and a deeper understanding of front-end architecture. Managing application state, routing, and component lifecycles in the browser can be more complex than traditional server-side rendering, leading to a steeper learning curve and potentially longer development cycles for larger teams.
- **Debugging:** Debugging issues that span the client-side MVC layers (e.g., data binding problems, unexpected view updates) can sometimes be more challenging than debugging simpler server-rendered pages.

Despite these trade-offs, for a modern, customer-centric Internet Banking platform that prioritises user responsiveness and a rich interactive experience, the Client-Side MVC pattern (as the structural foundation of an SPA) remains the most suitable choice, provided the architectural considerations, particularly security, are rigorously addressed.

Question 3:

The new platform must integrate with four existing systems presenting heterogeneous interfaces (Core Banking – REST, Inter-bank Payment Gateway – ISO 8583, Credit Card System – SOAP, Internal Payment Switch – TCP/IP POX).

a. Propose and justify the **primary architectural pattern** most suitable for handling these diverse and critical integrations within the overall solution.

b. Within your proposed integration pattern, identify and describe **three (3) specific Enterprise Integration Patterns (EIPs)** that would be essential for successfully connecting with these external systems. For each EIP, provide a concrete example of its application with one or more of the specified SriBank PLC external systems.

[Marks 25]

Answer 3:

Section (a)

Given SriBank PLC's scenario with four existing systems using diverse and critical interfaces (REST, ISO 8583, SOAP, TCP/IP POX), the primary architectural pattern most suitable for handling these integrations is an **Integration Layer** implemented with principles from **Enterprise Integration Patterns (EIPs)**, often manifested through an **Enterprise Service Bus (ESB)** or a more modern **API Management Platform/set of Integration Microservices**.

Primary Architectural Pattern: Integration Layer (enabled by ESB/API Management/Integration Microservices)

- **Description:** An Integration Layer is a dedicated architectural segment that facilitates communication and data exchange between disparate applications and systems. It acts as a central nervous system for enterprise-wide data flow. It abstracts the complexities of individual system interfaces, allowing applications within the core business logic to interact with external systems through a unified, standardised mechanism. This layer is crucial for managing heterogeneity in protocols, data formats, and communication styles.

- **Justification and Contribution to Requirements:**

  1. **Handling Heterogeneity:**

     - **Problem:** SriBank's scenario presents a classic integration challenge: Core Banking uses REST, the Inter-bank Payment Gateway uses ISO 8583, the Credit Card System uses SOAP, and the Internal Payment Switch uses TCP/IP POX. These are vastly different protocols, message formats, and communication paradigms. Direct point-to-point integration from the new banking platform to each of these would lead to a "spaghetti architecture" — a tangled mess of brittle, non-reusable connections.

     - **Solution:** The Integration Layer, through specialised adapters/connectors, centralises the logic for each protocol. It converts incoming requests from the new platform's internal, standardised format (e.g., RESTful calls) into the specific protocol and data format required by the target external system (e.g., ISO 8583, SOAP XML, POX XML over TCP/IP). This effectively shields the business logic tier from the complexities of these diverse interfaces.

  2. **Decoupling:**

     - **Problem:** Without an integration layer, the new Internet/Mobile Banking platform's business logic would be tightly coupled to the specific details

of each external system. A change in any external API (e.g., a field name in the Core Banking REST API, an update to the ISO 8583 specification) would require modifications and retesting within the core business logic, increasing maintenance burden and risk.

- **Solution:** The Integration Layer acts as an intermediary. The business logic tier interacts with a stable, internal interface exposed by the Integration Layer. If an external system's interface changes, only the corresponding adapter within the Integration Layer needs to be updated. This loose coupling enhances maintainability, reduces dependencies, and makes the system more resilient to external changes.

3. **Centralised Management and Monitoring:**

- **Problem:** Managing integrations across multiple systems without a centralised approach can lead to blind spots regarding data flow, error rates, and performance bottlenecks. Troubleshooting becomes a nightmare.

- **Solution:** A dedicated Integration Layer provides a single point for managing all external system connections. It allows for centralised logging, monitoring, and error handling for all integration flows. This improves visibility into system health, streamlines troubleshooting, and ensures robust operational oversight critical for a banking platform.

4. **Security Enhancement:**

- **Problem:** Exposing legacy systems or external payment gateways directly to an application can introduce security vulnerabilities if not properly managed. Each connection would require individual security considerations.

- **Solution:** The Integration Layer acts as a security enforcement point. It can handle authentication, authorisation, and encryption specific to each external system on behalf of the business logic. It can also abstract sensitive credentials or API keys away from the business logic tier, enhancing overall security posture by providing a controlled gateway to external resources.

5. **Reusability:**

- **Problem:** Specific integration logic (e.g., transforming data for the ISO 8583 gateway) might be needed by multiple internal services or even future applications. Without an integration layer, this logic might be duplicated.

- **Solution:** Components within the Integration Layer (e.g., the Core Banking Adapter) become reusable assets. Any part of the new Internet/Mobile Banking platform, or even future systems within SriBank, can leverage these pre-built, tested integration components, accelerating development and ensuring consistency.

In essence, the Integration Layer acts as a strategic abstraction that enables the new Internet and Mobile Banking Platform to seamlessly interact with SriBank's existing IT ecosystem, ensuring robustness, flexibility, and maintainability in a highly complex and critical environment.

Section (b)

1. **Splitter**

   - **Description:** The Splitter pattern takes a single, composite message and breaks it down into multiple, smaller, individual messages. Each of these smaller messages can then be processed independently. This is particularly useful when a single logical request from the client tier requires interaction with multiple backend systems or multiple operations within a single backend system.

   - **Application in SriBank PLC:** This pattern would be essential when a user action initiates a request that logically contains multiple distinct sub-requests that need separate processing by different (or the same) external systems.

     - **Example (Utility Bill Payment for Multiple Bills):**

       - **Problem:** The user might want to pay multiple utility bills (e.g., electricity, water, phone) in a single transaction from the Internet/Mobile Banking platform. The Retail Banking Service receives a composite Multi-Bill Payment Request message. However, the Inter-bank Payment Gateway or individual biller systems might only accept single bill payment requests or require different routing.

       - **Application of Splitter:**

         1. The Retail Banking Service (or a dedicated Payment Orchestrator component within the Integration Tier) receives a single Multi-Bill Payment Request containing a list of individual bills to pay.

         2. A **Splitter** component within the Integration Tier (e.g., as part of the Utility Bill Payment Orchestrator within the Integration Tier) would take this composite message and

break it into individual Single Bill Payment Request messages, one for each bill.

3. Each Single Bill Payment Request message can then be routed independently (potentially using a Content-Based Router, as discussed next) to the appropriate Inter-bank Payment Gateway Adapter or a specific biller's adapter for processing.

4. This allows the system to process each bill payment as a distinct transaction, increasing fault tolerance (failure of one bill payment doesn't necessarily block others) and enabling more granular logging and status tracking for each payment.

2. **Content-Based Router**

- **Description:** The Content-Based Router examines the content (payload or headers) of an incoming message and, based on predefined rules or criteria, routes the message to one or more specific output channels or endpoints. This pattern is crucial when different types of messages, or messages with specific characteristics, need to be handled by different processing logic or sent to different systems.

- **Application in SriBank PLC:** This pattern is fundamental for directing requests to the correct external system or sub-component based on the type or nature of the banking transaction.

  - **Example (Funds Transfer Routing):**

    - **Problem:** A Funds Transfer Request from the user could be for:

      1. An internal SriBank account (transfer within the bank).

      2. An account in another Sri Lankan bank (requiring the Inter-bank Payment Gateway).

      3. An international transfer (requiring the Internal Payment Switch for MasterCard/VISA).

    - The Retail Banking Service in the Business Logic Tier would initiate a generic TransferRequest.

    - **Application of Content-Based Router:**

1. A **Content-Based Router** component within the Integration Tier (perhaps within the Payment Orchestrator or Funds Transfer Service if it sits in the Integration Tier) receives the TransferRequest.
2. It examines the content of the message, specifically fields like toAccountBankIdentifier or transferType.
3. **Routing Rules:**
    - If toAccountBankIdentifier indicates an internal SriBank account, route the message to the Core Banking Adapter.
    - If toAccountBankIdentifier indicates a local external bank, route the message to the Inter-bank Payment Gateway Adapter.
    - If transferType is "International", route the message to the Internal Payment Switch Adapter.
4. This ensures that each type of transfer is directed to the correct specialised external system for processing, maintaining modularity and efficiency.

3. **Aggregator**
    - **Description:** The Aggregator pattern collects a set of related messages and combines them into a single, composite message. It's often used in conjunction with a Splitter or Scatter-Gather pattern, where a single request is fanned out to multiple recipients, and their responses need to be gathered and consolidated before returning a single result to the original requester. The Aggregator needs a correlation ID to identify which messages belong together and a completion condition to know when all expected messages have been received.
    - **Application in SriBank PLC:** This pattern would be vital for scenarios where the new platform needs to query multiple external systems to fulfil a single user request, or when processing responses from split messages.
        - **Example (Account Summary with Credit Card Details):**
            - **Problem:** A user requests an "Account Summary" which, for a comprehensive view, might need to include not just their deposit account balance from the Core Banking System but also their credit card outstanding balance and available limit from the

Credit Card System. These are two separate queries to two different external systems.

- **Application of Aggregator:**

  1. When the Retail Banking Service receives a GetAccountSummaryRequest that includes a request for credit card details, it can initiate two parallel (or sequential) sub-requests via the Integration Tier:

     - One request to the Core Banking Adapter for deposit account details.

     - Another request to the Credit Card System Adapter for credit card information.

  2. Each of these sub-requests is assigned a **correlation ID** linking it back to the original GetAccountSummaryRequest.

  3. An **Aggregator** component in the Integration Tier (or a dedicated AccountAggregator service) subscribes to the responses from both adapters based on the correlation ID.

  4. The Aggregator has a **completion condition** (e.g., "received responses from both Core Banking and Credit Card System").

  5. Once both responses are received, the Aggregator combines them into a single, comprehensive AccountSummaryResponse message, containing data from both systems.

  6. This aggregated response is then sent back to the Retail Banking Service and finally to the Presentation Tier for display to the user.

- This approach allows for efficient, potentially parallel, data retrieval from multiple sources, presenting a unified view to the user while managing the complexity of diverse backend calls.

These three EIPs — Splitter, Content-Based Router, and Aggregator — are fundamental tools within an Integration Layer to manage the complexities of heterogeneous systems, enabling flexible routing, comprehensive data processing, and efficient interaction with SriBank PLC's diverse external environments.

Question 4:

With specific reference to the notification functionality (push notifications and SMS) for mobile users in the SriBank PLC scenario:

a. Analyse and describe why **asynchronous communication** is the appropriate nature of interaction between the banking platform and the mobile applications for notifications, contrasting it briefly with synchronous communication.

b. Identify and describe the **most suitable architectural pattern** to implement this asynchronous notification functionality.

c. Explain how this architectural pattern would ensure **reliable and scalable delivery of notifications**, even when underlying systems or network conditions are volatile.

[Marks 25]

Answer 4:

Section (a)

For the SriBank PLC platform's notification functionality (push notifications and SMS), **asynchronous communication** is the unequivocally appropriate nature of interaction between the banking platform and the mobile applications. This choice addresses crucial non-functional requirements like scalability, reliability, and user experience that synchronous communication cannot effectively provide for notifications.

Understanding Synchronous vs. Asynchronous Communication:

- **Synchronous Communication:**
    - **Nature:** In synchronous communication, the sender initiates a request and *waits* for an immediate response from the receiver before it can proceed with its tasks. It's a blocking operation.
    - **Analogy:** A phone call. You speak, and you wait for the other person to respond before you continue your conversation. If the other person doesn't answer or drops the call, your conversation is blocked.
    - **Pros (for general use cases):** Simplicity of immediate feedback, direct cause-and-effect relationship, easy to understand the flow for simple request-response scenarios.
    - **Cons (especially for notifications):**
        - **Blocking:** The sending process (e.g., the transaction completion logic within the bank) would be blocked, waiting for confirmation that the notification has been delivered (not just sent) to the mobile device. This introduces unnecessary latency.
        - **Tight Coupling:** The sender becomes tightly coupled to the availability and performance of the notification delivery mechanism.
        - **Scalability Bottleneck:** If notification volume is high, the sender could become a bottleneck as it waits for each notification to be processed.
        - **Failure Propagation:** If the notification service or the mobile network is temporarily unavailable, the synchronous call would fail, potentially causing the main banking transaction process to fail or roll back, which is unacceptable for critical operations like funds transfers.

- **Asynchronous Communication:**
    - **Nature:** In asynchronous communication, the sender sends a message or triggers an event and does not wait for an immediate response. It continues with its other tasks. The response, if any, is handled separately or at a later time.
    - **Analogy:** Sending an email or a text message. You send it, and you don't wait for the recipient to reply before you do something else. The recipient will get it when they are available and respond at their convenience.
    - **Pros (specifically for notifications):**
        - **Non-Blocking and Improved Responsiveness:** When a banking platform process (e.g., a successful funds transfer) needs to trigger a notification, it simply "publishes" a notification event and immediately continues its core task (e.g., returning transaction confirmation to the user). It doesn't wait for the actual delivery to the mobile device. This ensures the main banking application remains highly responsive and does not suffer from delays caused by notification delivery.
        - **Decoupling:** The core banking services are decoupled from the complexities and potential unreliability of external notification delivery channels (mobile networks, APNS, FCM, SMS gateways). If an external notification service is temporarily down or slow, it doesn't impact the critical banking operations.
        - **Enhanced Scalability:** Notification requests can be queued (e.g., in a message broker). A dedicated notification service can process these queues at its own pace, independently scaling to handle spikes in notification volume. Multiple instances of the notification service can consume messages in parallel.
        - **Increased Reliability and Fault Tolerance:** Messages can be persisted in a queue. If the notification delivery service fails, messages are not lost and can be retried once the service recovers. This "at-least-once" delivery guarantee is crucial for banking notifications, ensuring customers are informed.
        - **Flexibility and Extensibility:** New notification channels (e.g., in-app messages, email alerts) can be added easily by introducing new subscribers to the notification events, without modifying the core banking logic.

Why Asynchronous is Appropriate for SriBank's Notifications:

Consider a critical event like a "Funds Transfer Completion":

1. **User Expectation:** The user expects an immediate confirmation of their funds transfer on the banking platform itself. They don't want this confirmation to be delayed because their phone might be offline, or the push notification service is having issues.

2. **Platform Reliability:** The banking platform's primary goal is to accurately process financial transactions. If the notification component fails for some reason, the core transaction should not fail.

3. **Scalability:** During peak hours, thousands of transactions might be completed simultaneously, each requiring a notification. A synchronous approach would create immense pressure on the notification delivery system, potentially leading to cascading failures or severe performance degradation for the entire platform. An asynchronous approach distributes this load over time and across dedicated, scalable components.

Therefore, the bank's core system "publishes" a TransactionCompletedEvent (asynchronously). A separate Notification Service (or components within it) "subscribes" to this event. This service then takes responsibility for delivering the push notification or SMS. If the mobile network is congested or the device is off, the notification service can retry later or mark it for delayed delivery, without affecting the immediate user experience on the banking platform itself.

In essence, asynchronous communication provides the resilience, scalability, and responsiveness necessary for a modern banking platform's notification system, safeguarding the core transaction processing from the inherent variabilities and potential unreliability of external mobile delivery channels.

Section (b)

Most Suitable Architectural Pattern for Notification Functionality: Publish–Subscribe (Pub/Sub)

The **Publish–Subscribe (Pub/Sub)** architectural pattern is the most suitable and widely adopted pattern for implementing the asynchronous notification functionality required by SriBank PLC. It perfectly addresses the need for decoupled, scalable, and reliable message delivery for push notifications and SMS.

Description of the Publish-Subscribe (Pub/Sub) Pattern:

The Publish-Subscribe pattern defines a loose coupling between "publishers" (senders) and "subscribers" (receivers) through an intermediary component called a **Message Broker (or Event Broker)**.

- **Publishers:** Components that create and send messages (often called "events" or "notifications") to a specific topic or channel on the Message Broker. Publishers are unaware of who (if anyone) will consume these messages. They simply publish an event when something significant happens.
- **Subscribers:** Components that express interest in specific topics or message types. They "subscribe" to these topics on the Message Broker and receive all messages published to them. Subscribers are unaware of who published the message.
- **Message Broker:** The central component that facilitates communication. It receives messages from publishers, filters them by topic, and delivers them to all interested subscribers. The broker is responsible for reliable message delivery, often including message persistence, queuing, and acknowledgement mechanisms.

The key characteristic of Pub/Sub is the **decoupling** of producers (publishers) from consumers (subscribers). They communicate indirectly via the message broker, promoting flexibility, scalability, and resilience.

How Publish-Subscribe Addresses SriBank PLC's Notification Requirements:

Let's illustrate how Pub/Sub directly fulfils the needs for asynchronous, reliable, and scalable notification delivery in the SriBank scenario:

1. **Enabling Asynchronous Communication (Decoupling):**
    - **Scenario Application:** When a critical event occurs within the banking platform (e.g., a successful FundsTransfer, a LoanApproved status, or an AccountCreated confirmation), the relevant service (e.g., Retail Banking Service, Special Service Request Service) acts as a **publisher**.
    - **Mechanism:** Instead of making a direct, blocking call to a notification delivery service, the publisher simply creates an event message (e.g., FundsTransferCompletedEvent containing transaction details, user ID, etc.) and sends it to a designated topic on the **Message Broker** (e.g., notification_events_topic).
    - **Benefit:** The publisher immediately completes its primary task (e.g., confirming the transaction to the user on the screen) and does not wait for the notification to be processed or delivered. This ensures the core banking operations remain

responsive and unblocked, even if the notification delivery pipeline experiences delays or temporary outages.

2. **Reliability and Guaranteed Delivery (through the Message Broker):**

   - **Scenario Application:** Banking notifications are critical; customers must receive alerts about their transactions. Failures in delivery cannot be tolerated silently.

   - **Mechanism:** The **Message Broker** (e.g., Kafka, RabbitMQ) is designed for reliability.

     - When a publisher sends an event, the broker persists the message to disk (or memory with replication) before acknowledging receipt to the publisher. This ensures the message is not lost even if the broker itself crashes.

     - Subscribers (the Notification Delivery Service) pull messages from the broker. The broker tracks which messages have been successfully processed and acknowledged by the subscriber. If a subscriber fails mid-processing, the message is not lost and can be redelivered to another instance of the subscriber or retried later.

   - **Benefit:** This ensures "at-least-once" delivery semantics. Even if the Notification Delivery Service is temporarily down or external push notification gateways are experiencing issues, the messages queue up safely in the broker and will be processed once the service recovers, ensuring eventual delivery and high reliability.

3. **Scalability (Independent Scaling):**

   - **Scenario Application:** SriBank operates across 100+ branches with a large customer base. During peak hours, a high volume of transactions could generate a massive surge in notification requests.

   - **Mechanism:**

     - **Publishers:** Can scale independently. As more transactions occur, more instances of core banking services can publish events without needing to know or overload the notification system directly.

     - **Subscribers:** The Notification Delivery Service (the subscriber) can be scaled horizontally. If notification volume increases, more instances of this service can be deployed to consume messages from the broker in parallel. The message broker distributes the load among these instances.

        - **Message Broker:** The broker itself is built for high throughput and can handle large volumes of messages, acting as a buffer for spikes in traffic.

    - **Benefit:** This elasticity ensures the notification system can handle fluctuating loads without compromising the performance of the core banking platform or leading to backlogs that degrade user experience.

4. **Flexibility and Extensibility:**

    - **Scenario Application:** SriBank currently requires push notifications and SMS. In the future, they might want to add email alerts, in-app messaging, or even voice notifications.

    - **Mechanism:** With Pub/Sub, adding a new notification channel is straightforward. You simply deploy a new **subscriber** service that listens to the same notification_events_topic and implements the logic for the new channel (e.g., an EmailNotificationService). The publishers (core banking services) do not need to be modified.

    - **Benefit:** This allows SriBank to easily extend its notification capabilities in the future without impacting existing functionality or requiring downtime for core systems.

In summary, the Publish-Subscribe pattern, orchestrated by a robust Message Broker, provides the ideal foundation for SriBank PLC's notification functionality by ensuring asynchronous operation, high reliability, seamless scalability, and flexible extensibility, all of which are paramount for a customer-centric and critical banking platform.

Section (c)

The **Publish-Subscribe (Pub/Sub)** architectural pattern, typically implemented using a **Message Broker**, is supremely effective in ensuring reliable and scalable delivery of notifications, even when underlying systems (like the notification delivery service itself) or network conditions (to mobile devices) are volatile. This is primarily due to its inherent design principles of decoupling, asynchronous processing, and the robust features of the message broker.

Let's break down how Pub/Sub achieves this reliability and scalability:

Ensuring Reliability:

Reliability in the context of notifications means that once an event (like a funds transfer completion) occurs, a corresponding notification should eventually be delivered to the intended recipient, even if there are temporary failures or delays in the delivery pipeline.

1. **Decoupling of Publisher and Subscriber (Time and Space Decoupling):**

- **How it works:** The core banking services (publishers) that generate notification events (e.g., Retail Banking Service publishing a TransactionCompletedEvent) do not directly call the mobile notification service. Instead, they send the event to a neutral **Message Broker**. The Notification Delivery Service (subscriber) independently consumes messages from this broker.

- **Impact on Volatility:**

  - **No Blocking:** If the Notification Delivery Service is temporarily unavailable (e.g., due to a deployment, a crash, or an external push gateway being down), the publishing banking service is not blocked. It successfully delivers the message to the broker and can continue processing other critical banking transactions. This prevents notification delivery issues from impacting core system functionality.

  - **Tolerance to Subscriber Downtime:** The message broker acts as a buffer. Messages published while a subscriber is offline or unavailable are held by the broker. Once the subscriber comes back online, it can resume processing the backlog of messages from where it left off. This ensures notifications are not lost due to temporary system outages.

2. **Message Persistence and Durability (Broker's Role):**

   - **How it works:** Modern message brokers (e.g., Kafka, RabbitMQ, Azure Service Bus, AWS SQS/SNS) are designed to persist messages to disk (or highly available replicated storage) immediately upon receipt from a publisher, before acknowledging the message to the publisher.

   - **Impact on Volatility:**

     - **No Message Loss:** Even if the message broker itself crashes or restarts, the messages that have been received and persisted are not lost. They remain in the queue or topic log until a subscriber successfully processes them. This is critical for banking, where every notification (e.g., a credit alert) is important.

     - **"At-Least-Once" Delivery:** The broker typically implements acknowledgement mechanisms. A message is only removed from the broker's queue/topic once the subscriber explicitly confirms (acknowledges) that it has successfully processed and delivered the notification. If no acknowledgement is received within a timeout, the message is assumed to have failed delivery and is redelivered (potentially to another subscriber instance). This guarantees that notifications are delivered.

3. **Dead-Letter Queues (DLQs) and Error Handling:**

   - **How it works:** Many message brokers support Dead-Letter Queues (DLQs). If a message cannot be processed by the subscriber after a certain number of retries (e.g., due to an invalid message format, persistent error in the external API), it is moved to a DLQ instead of being discarded.

   - **Impact on Volatility:**

     - **Error Isolation:** This prevents "poison pill" messages from perpetually blocking a subscriber.

     - **Auditability & Recovery:** Messages in the DLQ can be inspected by operations teams, debugged, fixed, and then potentially re-injected into the main queue for reprocessing. This provides a mechanism for manual intervention and recovery for notifications that encounter unrecoverable errors.

Ensuring Scalability:

Scalability in this context means the ability of the notification system to handle increasing volumes of notifications and a growing number of mobile users without degrading performance or requiring significant architectural overhauls.

1. **Asynchronous Nature and Load Smoothing:**

   - **How it works:** Publishers are not waiting for subscribers. They send events and move on. This allows the core banking system to generate notifications at a very high rate without being constrained by the speed of notification delivery.

   - **Impact on Volatility:**

     - **Buffer for Spikes:** The message broker acts as a buffer. If there's a sudden surge of transactions (e.g., end-of-month bill payments) leading to a high volume of notifications, the broker can queue these messages. The Notification Delivery Service can process them at its maximum sustainable rate, rather than being overwhelmed. This smooths out load peaks.

2. **Horizontal Scaling of Subscribers:**

   - **How it works:** The Pub/Sub pattern enables easy horizontal scaling of consumers. The Notification Delivery Service can be deployed as multiple instances, all subscribing to the same notification topic. The message broker automatically distributes the incoming messages among these available instances (often using consumer groups), allowing them to process messages in parallel.

- **Impact on Volatility:**
  - **Increased Throughput:** As notification volume grows, more instances of the Notification Delivery Service can be added to increase processing throughput. This elastic scaling capability means the system can adapt to increasing demand without requiring changes to the publishers.
  - **High Availability:** If one instance of the Notification Delivery Service fails, the message broker will automatically reassign its pending messages to other healthy instances, ensuring continuous processing and high availability of the notification functionality.

3. **Independent Evolution:**
   - **How it works:** Because publishers and subscribers are decoupled and communicate through well-defined message formats, they can evolve independently.
   - **Impact on Volatility:**
     - **Reduced Regression Risk:** Changes to how notifications are delivered (e.g., upgrading to a new version of the push notification API, integrating a new SMS provider) can be done entirely within the Notification Delivery Service without affecting the core banking services that publish events. This minimises the risk of introducing bugs into the core transaction processing system.

In essence, the Publish-Subscribe architecture, underpinned by a robust message broker, transforms potentially brittle, tightly coupled notification delivery into a resilient, highly available, and horizontally scalable asynchronous system. This design is crucial for a banking platform like SriBank PLC, ensuring critical information reaches customers reliably even amidst the inherent volatilities of networks and external systems.

"Tell me and I forget. Teach me and I remember. Involve me and I learn."

Benjamin Franklin

[END]